

PEC2 - Segunda prueba de evaluación continua

Presentación

Después de conocer los sistemas de representación de la información, esta PEC se focaliza en los circuitos lógicos combinacionales. Las funciones lógicas nos permiten describir la funcionalidad de un circuito, y mediante las puertas lógicas y los bloques combinacionales podemos implementarlas en un circuito. Todo esto no se podría hacer sin conocer mecanismos de minimización, como mapas de Karnaugh, que nos permiten reducir las dimensiones del circuito que se tiene que implementar.

Competencias

- Conocer y saber aplicar el álgebra de Boole para la manipulación de funciones lógicas.
- Tener nociones tecnológicas de los circuitos digitales y entender la relación entre los circuitos digitales y las funciones lógicas.
- Conocer y saber utilizar las puertas lógicas y los módulos combinacionales en el diseño de circuitos lógicos.

Objetivos

- Saber aplicar las operaciones básicas y los axiomas del álgebra de Boole.
- Saber representar las funciones lógicas mediante tablas de verdad.
- Saber representar las funciones lógicas mediante expresiones algebraicas.
- Saber analizar un circuito combinacional.
- Saber realizar un cronograma a partir de un circuito digital combinacional.
- Saber sintetizar una función a dos niveles.
- Saber diseñar un circuito combinacional sencillo a partir de los bloques combinacionales de los materiales.

Recursos

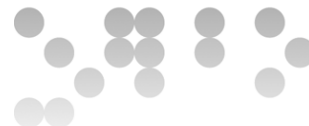
Los recursos que se recomienda usar para esta PEC son los siguientes:

Básicos: El módulo 3 de los materiales.

Complementarios: KeMap, VerilCIRC, VerilChart y el Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.
- La valoración está indicada en cada uno de los subapartados.



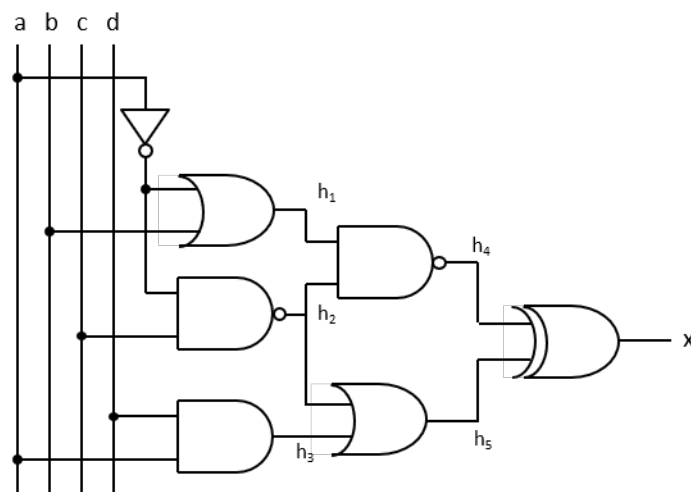
Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigiós al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF usando una de las plantillas publicadas juntamente con este enunciado.
- Se tiene que entregar a través de la aplicación de **Entrega y registro de EC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **30 de marzo** (a las 24 horas).

Enunciado de la PEC

Ejercicio 1 [20%]

Dado el circuito lógico combinacional siguiente:



Rellenad la tabla de verdad que especifica la salida x en función de las entradas a , b , c y d . Calculad previamente los valores intermedios (h_1 , h_2 , h_3 , h_4 , h_5) indicados en el circuito y añadidlos en la tabla de verdad siguiente:

Del circuito podemos extraer las siguientes expresiones:

$$h_1 = a' + b$$

$$h_2 = (a' \cdot c)'$$

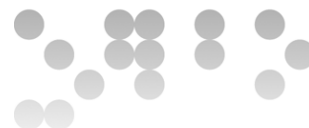
$$h_3 = a \cdot d$$

$$h_4 = (h_1 \cdot h_2)'$$

$$h_5 = h_2 + h_3$$

$$x = h_4 \oplus h_5$$

Y por tanto, la tabla queda de la siguiente manera:

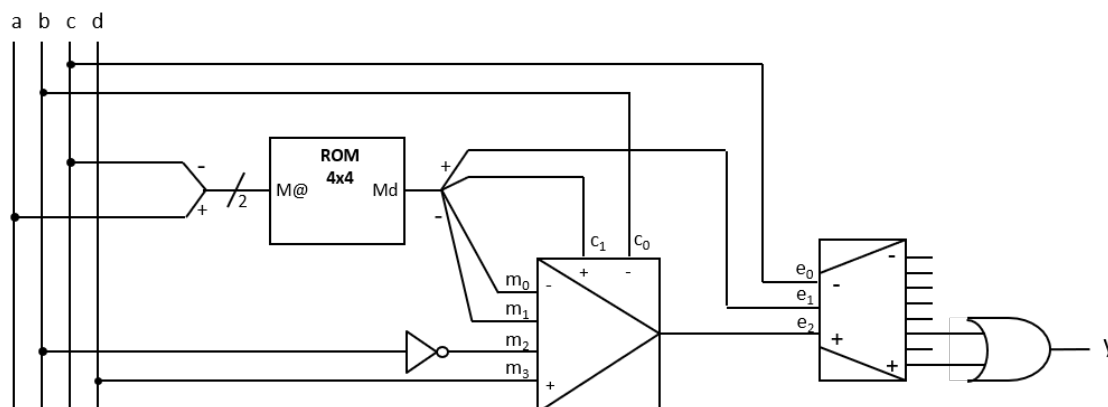


a	b	c	d	h_1	h_2	h_3	h_4	h_5	x
0	0	0	0	1	1	0	0	1	1
0	0	0	1	1	1	0	0	1	1
0	0	1	0	1	0	0	1	0	1
0	0	1	1	1	0	0	1	0	1
0	1	0	0	1	1	0	0	1	1
0	1	0	1	1	1	0	0	1	1
0	1	1	0	1	0	0	1	0	1
0	1	1	1	1	0	0	1	0	1
1	0	0	0	0	1	0	1	1	0
1	0	0	1	0	1	1	1	1	0
1	0	1	0	0	1	0	1	1	0
1	0	1	1	0	1	1	1	1	0
1	1	0	0	1	1	0	0	1	1
1	1	0	1	1	1	1	0	1	1
1	1	1	0	1	1	0	0	1	1
1	1	1	1	1	1	1	0	1	1

NOTA: Tenéis disponible el ejercicio en VerilUOC para verificarlo. Para evitar un mal uso de VerilUOC, en el caso de que se hagan más de 5 intentos la nota del ejercicio se reducirá al 50%.

Ejercicio 2 [20%]

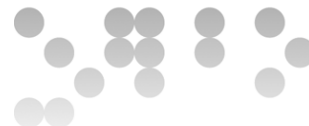
Dado el circuito lógico combinacional siguiente:



Donde el contenido de la ROM es el siguiente:

M@	Md
0	1010
1	1100
2	0110
3	0101

Completad la tabla de verdad que especifica la salida y en función de las entradas a , b , c y d . Calculad previamente los valores intermedios (m_0 , m_1 , m_2 , m_3 , c_0 , c_1 , e_0 , e_1 , e_2) indicados en el circuito y añadidos a la tabla de verdad siguiente:



Para obtener la salida y empezaremos obteniendo la entrada de la memoria ROM. La entrada que indica la dirección de memoria viene directamente dada por los valores de a y c . En la salida de la ROM tendremos $e_1c_1m_0m_1$. Por lo tanto, cuando a y c valen 0, apuntaremos a la primera dirección de memoria ROM que tiene el valor "1010" y entonces $e_1=1$, $c_1=0$, $m_0=1$, $m_1=0$. Cuando $a=0$ y $c=1$, la salida de la memoria es "1100", y entonces $e_1=1$, $c_1=1$, $m_0=0$, $m_1=0$. Cuando $a=1$ y $c=0$, la salida de la memoria es "0110" y entonces $e_1=0$, $c_1=1$, $m_0=1$ y $m_1=0$. Por último, cuando $a=1$ y $c=1$, la salida de la memoria es "0101" y por tanto, $e_1=0$, $c_1=1$, $m_0=0$ y $m_1=1$.

Las entradas del MUX son m_0 y m_1 , las cuales obtenemos del valor de la ROM, y $m_2=b'$ y $m_3=d$. Los bits de selección del MUX son c_1 , que lo obtenemos del valor de salida de la ROM, y $c_0=b$. De aquí obtenemos el valor de salida e_2 del descodificador, y además e_1 , lo obtenemos de la salida de la ROM y $e_0=c$.

Por último, en la salida del descodificador tenemos que el bit 7 y el bit 5 acaban en una puerta OR. Por lo tanto, $y = s_7 + s_5$. y sólo será 1 cuando en la entrada del descodificador tengamos un 5 o un 7.

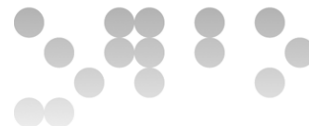
Con todo esto ya podemos completar toda la tabla:

a	b	c	d	m_0	m_1	m_2	m_3	c_1	c_0	e_0	e_1	e_2	y
0	0	0	0	1	0	1	0	0	0	0	1	1	0
0	0	0	1	1	0	1	1	0	0	0	1	1	0
0	0	1	0	0	0	1	0	1	0	1	1	1	1
0	0	1	1	0	0	1	1	1	0	1	1	1	1
0	1	0	0	1	0	0	0	0	1	0	1	0	0
0	1	0	1	1	0	0	1	0	1	0	1	0	0
0	1	1	0	0	0	0	0	1	1	1	1	0	0
0	1	1	1	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	0	1	0	0	0	1	0
1	0	0	1	1	0	1	1	1	0	0	0	1	0
1	0	1	0	0	1	1	0	1	0	1	0	1	1
1	0	1	1	0	1	1	1	1	0	1	0	1	1
1	1	0	0	1	0	0	0	1	1	0	0	0	0
1	1	0	1	1	0	0	1	1	1	0	0	1	0
1	1	1	0	0	1	0	0	1	1	1	0	0	0
1	1	1	1	0	1	0	1	1	1	1	0	1	1

NOTA: Tenéis disponible el ejercicio en VerilUOC para verificarlo. Para evitar un mal uso de VerilUOC, en el caso de que se hagan más de 5 intentos la nota del ejercicio se reducirá al 50%.

Ejercicio 3 [20%]

Dada la tabla de verdad siguiente:



a	b	c	d	f ₁	f ₂
0	0	0	0	0	1
0	0	0	1	0	0
0	0	1	0	1	0
0	0	1	1	0	1
0	1	0	0	0	X
0	1	0	1	0	X
0	1	1	0	0	X
0	1	1	1	0	X
1	0	0	0	1	X
1	0	0	1	0	X
1	0	1	0	1	1
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	1	1	0
1	1	1	0	0	X
1	1	1	1	1	X

- a) **[10%]** Sintetizad de manera mínima a dos niveles la función f_1 mediante el método de Karnaugh, e implementad el resultado con un circuito con puertas lógicas.

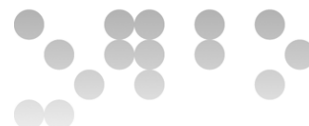
NOTA: Tenéis disponible el ejercicio en KeMAP para verificarlo. En cuanto al mapa de Karnaugh, para evitar un mal uso de KeMAP, en caso de que se hagan más de 5 intentos la nota del ejercicio se reducirá al 50 %. Os recomendamos hacer pruebas con KeMAP con otros ejercicios antes de hacer los de la PEC.

El mapa de Karnaugh de la función f_1 es el siguiente:

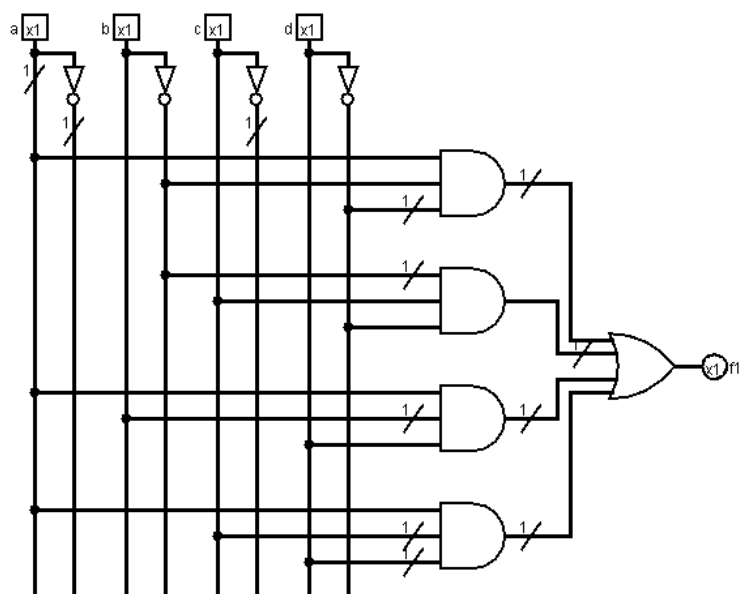
ab \ cd	00	01	11	10
00	0	0	0	1
01	0	0	1	0
11	0	0	1	1
10	1	0	0	1

Y obtenemos esta expresión mínima:

$$f_1 = a \cdot b' \cdot d' + b' \cdot c \cdot d' + a \cdot b \cdot d + a \cdot c \cdot d$$



El circuito que implementa esta función con puertas lógicas es el siguiente:



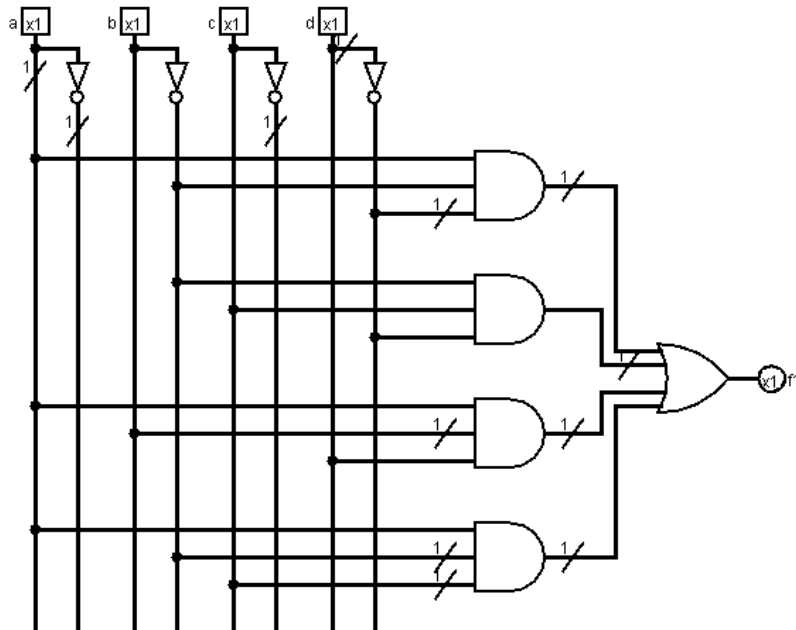
Una solución alternativa del mapa de Karnaugh de la función f_1 es el siguiente:

ab \ cd	00	01	11	10
00	0	0	0	1
01	0	0	1	0
11	0	0	1	1
10	1	0	0	1

Y obtenemos esta expresión mínima:

$$f_1 = a \cdot b' \cdot d' + b' \cdot c \cdot d' + a \cdot b \cdot d + a \cdot b' \cdot c$$

El circuito que implementa esta función con puertas lógicas es el siguiente:



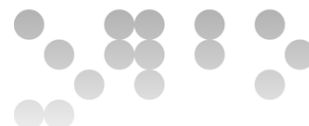
- b) **[10%]** Sintetizad de manera mínima a dos niveles la función f_2 mediante el método de Karnaugh, e implementad el resultado con un circuito con puertas lógicas.

NOTA: Tenéis disponible el ejercicio en KeMAP para verificarlo. En cuanto al mapa de Karnaugh, para evitar un mal uso de KeMAP, en caso de que se hagan más de 5 intentos la nota del ejercicio se reducirá al 50 %. Os recomendamos hacer pruebas con KeMAP con otros ejercicios antes de hacer los de la PEC.

El mapa de Karnaugh de la función f_2 es el siguiente:

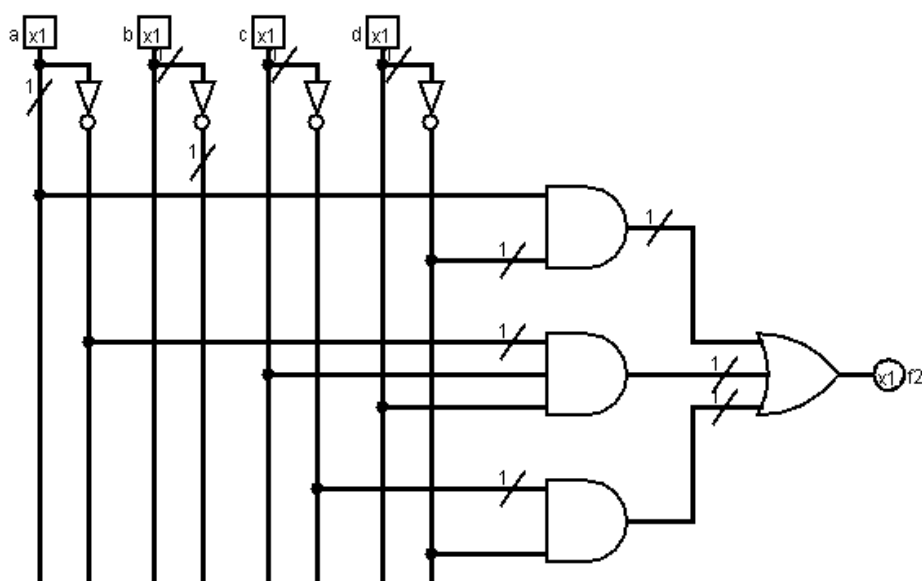
ab \ cd	00	01	11	10
00	1	X	1	X
01	0	X	0	X
11	1	X	X	0
10	0	X	X	1

Y obtenemos esta expresión mínima:



$$f_1 = a \cdot d' + a' \cdot c \cdot d + c' \cdot d'$$

El circuito que implementa esta función con puertas lógicas es el siguiente:

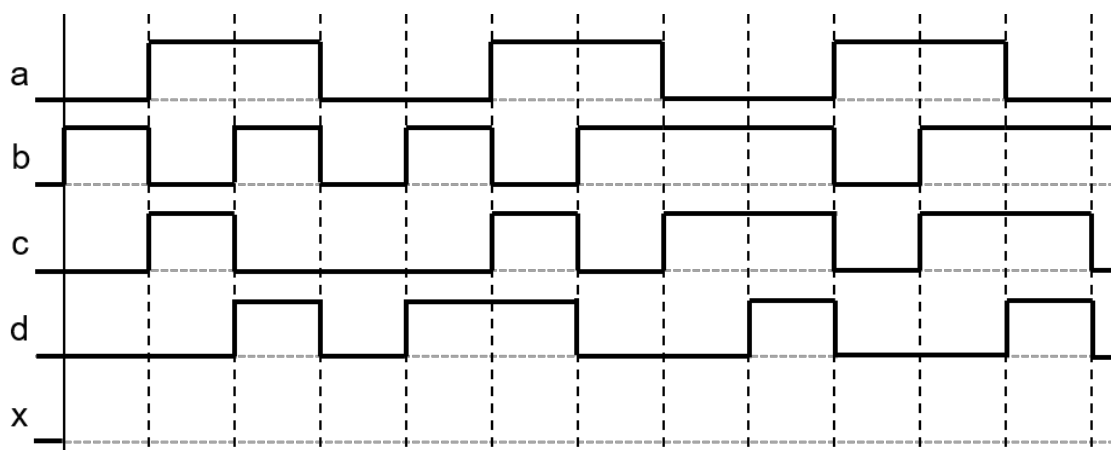
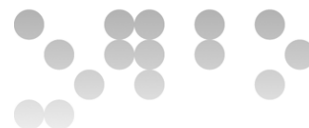


Ejercicio 4 [20%]

a) [10%] Dada la tabla de verdad siguiente:

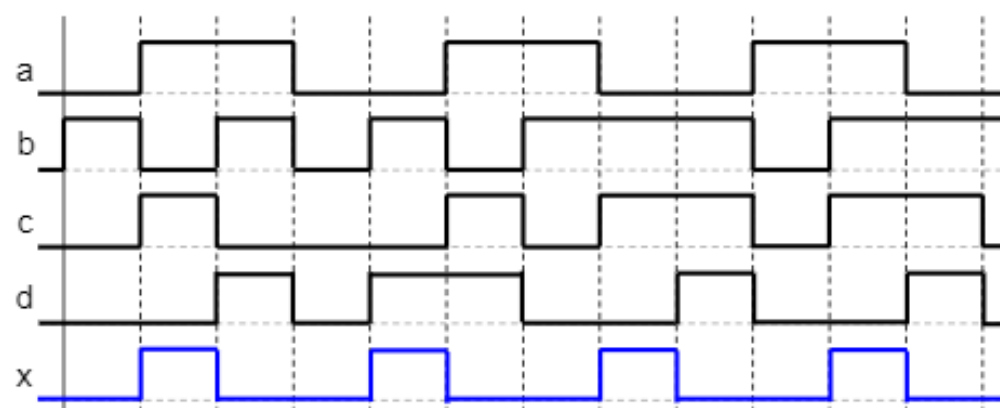
a	b	c	d	x
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

Completad el cronograma siguiente:

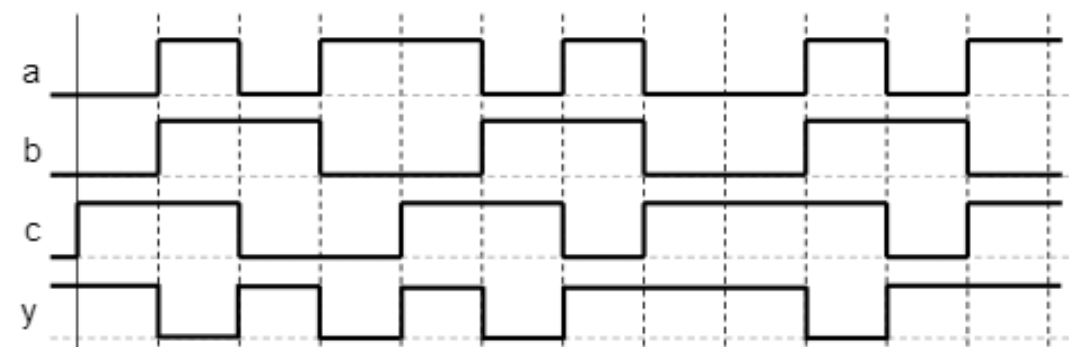


NOTA: Tenéis disponible el ejercicio en VerilCHART.

A partir de los valores de la tabla obtenemos el siguiente cronograma:

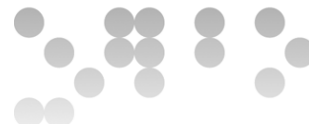


b) [10%] Dado el cronograma siguiente:



Completad tabla de verdad siguiente:

a	b	c	y
0	0	0	
0	0	1	



0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

A partir de los valores del cronograma obtenemos la siguiente tabla:

a	b	c	Y
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Ejercicio 5 [20%]

- a) [5%] Se quiere diseñar un circuito combinacional denominado *PRIM* de acuerdo con la estructura siguiente:

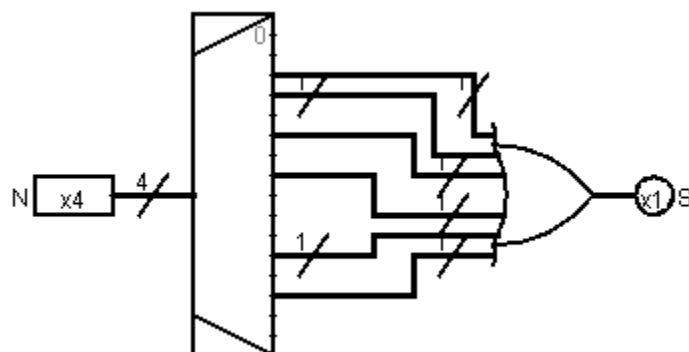
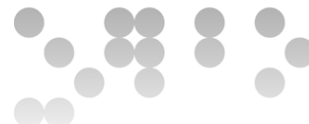


donde la entrada N corresponde a un número binario natural de 4 bits. La salida S , de un bit, tiene que valer 1 si el número N es un número primo, o 0 en cualquier otro caso.

Diseñad el circuito *PRIM* usando un decodificador de 4 entradas y las puertas lógicas que sean necesarias.

Como que N es un número de 4 bits tomará valores en el rango $[0,15]$. Se consideran números primos el 2, 3, 5, 7, 11 y 13. Por lo tanto, cuando la entrada N tenga estos valores la salida S tendrá que ser 1, y en el resto de casos tendrá que ser 0. Esto lo podemos resolver haciendo que N sea la entrada del codificador, y conectando las salidas 2, 3, 5, 7, 11 y 13 del decodificador a una puerta OR. La salida de la puerta OR será directamente S .

NOTA: A pesar de que por convenio el número 1 no se considera número primo, si un alumno realiza la solución cogiendo también el 1 se puede considerar que se soluciona correctamente el ejercicio.



NOTA: Tenéis disponible el ejercicio en VerilCIRC, sin límite en el número de intentos para comprobar vuestra solución.

- b) **[15%]** Se quiere diseñar un circuito combinacional denominado *DATA*, con una entrada *N* de 4 bits que corresponde a un número binario natural y una salida *X* de 4 bits, de acuerdo con la estructura siguiente:



El circuito se tiene que comportar tal y como se describe a continuación:

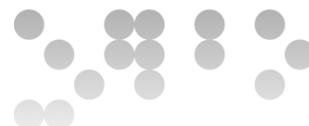
- Si el número *N* es un número primo, entonces la salida *X* tiene que ser el mismo número *N*;
- Si no, si el número *N* es un número par, entonces la salida *X* tiene que ser $N / 2$;
- De lo contrario, el valor de la salida *X* tiene que ser $N \cdot 3 / 4$. Tened en cuenta que la división de un número natural por 4 es una división sin decimales, por truncamiento.

Usando el circuito *PRIM* obtenido en el apartado anterior, se pide que diseñéis el circuito *DATA*.

NOTA: Tenéis disponible el ejercicio en VerilCIRC, sin límite en el número de intentos para comprobar vuestra solución.

Por saber si *N* es par o no, cogemos el bit menos significativo de *N*. Si este es 0 significa que *N* es par y si es 1, *N* es impar. Este bit irá a un MUX para escoger entre $N/2$ si es par o $N \cdot 3/4$ si es impar.

Para hacer $N/2$ hacemos un desplazamiento a la derecha, y para encontrar $N \cdot 3/4$ hacemos un sumatorio de *N* más $2N$ (desplazando a la izquierda) y el resultado lo dividimos entre 4 (desplazamos a la derecha 2 bits) $((N+2N)/4)$.



Para encontrar si N es un número primo utilizamos el circuito del apartado anterior, y la salida la llevaremos a un MUX para obtener como salida N si S es 1 o la salida del otro MUX si S es 0.

Así pues, obtenemos el circuito siguiente:

